

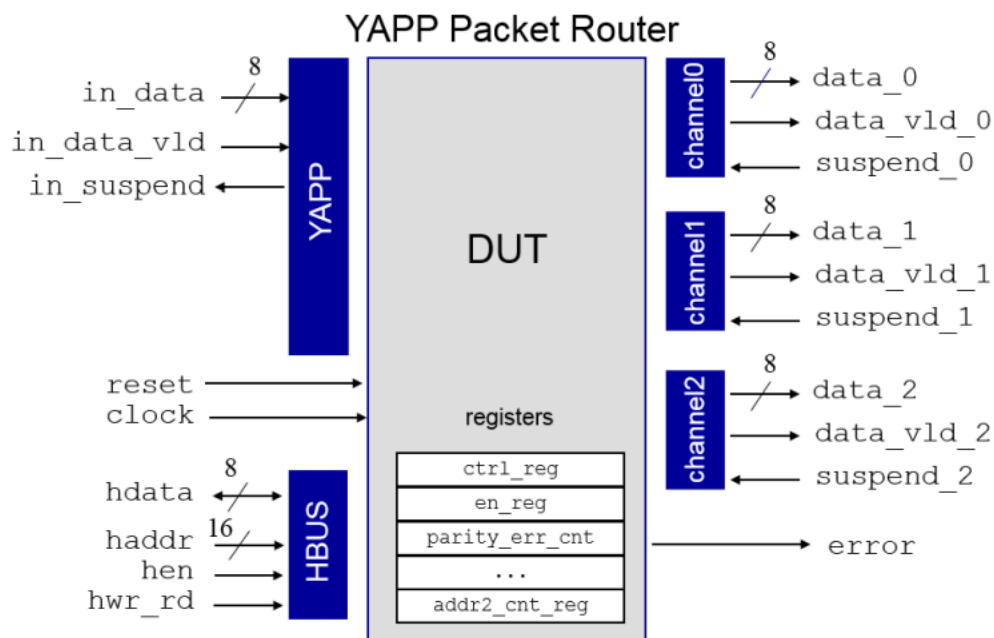
Router

Verification Plan Document

<Chaya Zander >

<03-11-2025 / rev 1.0>

1 DUT Overview



The DUT is a YAPP Packet Router that receives packets via the YAPP interface, parses the header to determine the output channel, and routes the packet to one of three output channels (channel0, channel1, channel2). The router supports error detection (parity, length, illegal address), maintains counters and memories, and is programmable via the HBUS host interface.

1.1 Clocks

- The router operates on a single clock signal.
- All input and output signals are sampled/driven on the falling edge of the clock.

1.2 Resets

- The router supports asynchronous reset.
- On reset, all registers, counters, FIFOs, and memories are initialized to default values.

1.3 More Declarations

- Data width: 8 bits
- Address width: 2 bits (in header)
- Length field: 6 bits (in header)
- FIFO depth per channel: 16 bytes
- HBUS interface: hdata (8 bits), haddr (16 bits), hen, hwr_rd
- Packet structure: header (address + length), payload (1–63 bytes), parity (1 byte)

2 Design Assumptions and Configuration

2.1 Assumptions

- All counters and registers are initialized to default values at the beginning of each test
- No sudden changes in voltage or clock frequency occur during testing
- All input and output signals are synchronized to the negative edge of the clock
- No register writes or modifications through HBUS interface occur during packet processing
- YAPP packet structure conforms to the official specification
- Input data is assumed to be valid with no bit-level errors

2.2 Configuration

address	register	reset	field	field name	policy	description
0x1000	ctrl_reg	0x3f	5:0	maxpktsize	RW	Maximum packet length
			7:6		RW	Unused
0x1001	en_reg	0x01	0	router_en	RW	Router enable
			1	parity_err_cnt_en	RW	Parity error count enable
			2	oversized_pkt_cnt_en	RW	Length error count enable
			3	[reserved]	RW	Not implemented
			4	addr0_cnt_en	RW	Address 0 packet count enable
			5	addr1_cnt_en	RW	Address 1 packet count enable
			6	addr2_cnt_en	RW	Address 2 packet count enable
			7	addr3_cnt_en	RW	Address 3 packet count enable
0x1004	parity_err_cnt_reg	0x00	7:0		RO	Packet parity error count
0x1005	oversized_pkt_cnt_reg	0x00	7:0		RO	Packet length error count
0x1006	addr3_cnt_reg	0x00	7:0		RO	Address 3 packet count
0x1009	addr0_cnt_reg	0x00	7:0		RO	Address 0 packet count
0x100a	addr1_cnt_reg	0x00	7:0		RO	Address 1 packet count
0x100b	addr2_cnt_reg	0x00	7:0		RO	Address 2 packet count

Start Address	Name	Size	Policy	Description
0x1010	yapp_pkt_mem	[0:63]	RO	Stores the bytes of the last packet received by the yapp router.
0x1100	yapp_mem	[0:255]	RW	"Scratch" memory

3 Features (Design expected behavior)

1. The RTL shall route packets from in_data to channel0/1/2 by header address; addr=3 is illegal and dropped.
2. The RTL shall parse header: [7:6]=address, [5:0]=length.
3. The RTL shall accept payload length 1-63; otherwise drop packet.
4. The RTL shall enforce maxpktsize and drop oversized packets.
5. The RTL shall check parity over header+payload and assert error within 1-10 cycles if mismatch.
6. The RTL shall sample input only when in_data_vld=1 on the specified clock edge.
7. The RTL shall assert in_suspend when input FIFO is full.

8. The RTL shall assert `data_vld_x` when valid output is available on channel `x`.
9. The RTL shall implement per-channel FIFOs (depth 16, width 8 bits).
10. The RTL shall implement HBUS registers: `ctrl_reg`, `en_reg`, counters, memories per spec.
11. The RTL shall increment counters (errors, `addr0-3`, oversized) only when enabled in `en_reg`.
12. The RTL shall follow HBUS timing: write = 1 cycle, read = 2 cycles.
13. The RTL shall drop all packets when `router_en=0`.
14. The RTL shall expose internal memories: `yapp_pkt_mem` (last packet) and `yapp_mem` (scratch).
15. The RTL shall provide observable diagnostics for dropped/errored packets.
16. The RTL shall reset registers, counters, and FIFOs to default values on reset.

4 Flows & Cases production (test cases)

4.1 Per configuration:

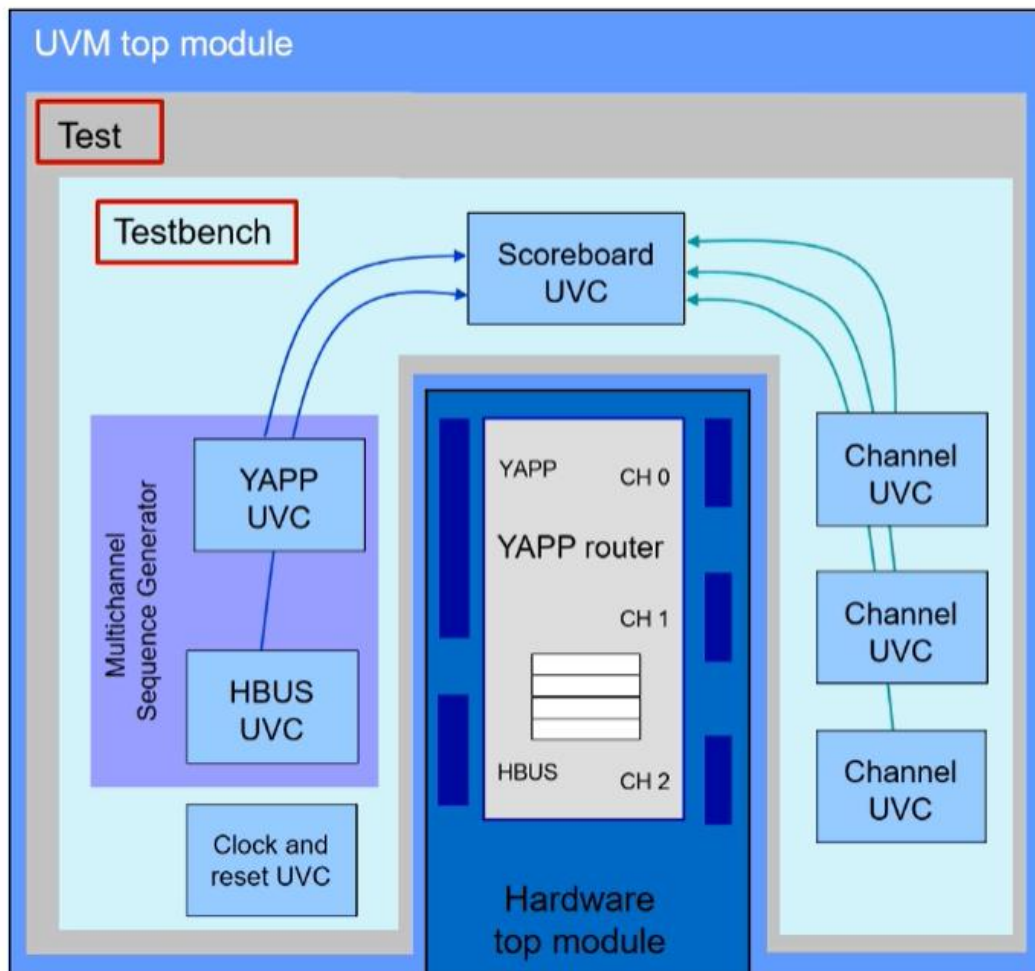
- Send valid packets to each channel and verify correct routing
- Send packets with `addr=3` and verify they are dropped
- Send packets with payload length `<1` or `>63` and verify they are dropped
- Send oversized packets (length `> maxpktsize`) and verify error counter increments
- Send packets with parity errors and verify error signal and counter
- Disable router (`router_en=0`) and verify all packets are dropped
- Enable/disable error counters and verify correct counting behavior
- Perform HBUS register read/write and verify timing and data integrity
- Assert reset during operation and verify all internal states return to default

4.2 Run-time (on the fly)

- Randomized packet injection with varying addresses, lengths, and parity
- Burst traffic to fill FIFOs and trigger suspend/in_suspend behavior
- Simulate HBUS accesses during packet processing (should be ignored per assumptions)
- Mix valid and invalid packets to stress error handling and diagnostics
- Change configuration (e.g., `maxpktsize`) during simulation and observe router response

4.3 Bring-up stages

5 Environment



router_env – top-level UVM environment that instantiates YAPP, HBUS, and Channel agents, the scoreboard, and the reference model. Manages configuration propagation and TLM connections.

5.1 YAPP Agent

This agent is responsible for the input interface of the router.

5.1.1 I/F

yapp_if include: in_data, in_data_vld, in_suspend, clock, reset.

5.1.2 Config - Agent Configuration

This block defines the main configuration parameters used by the other environment components:

- Activation mode – determine whether the agent operates in active or passive mode.
- Parity error injection enable - allows controlled insertion of parity errors during simulation
- Maximum packet size - parameter linked to ctrl_reg. maxpktsize.
- Reset behavior - defines agent behavior during and after reset.

This configuration is defined per test.

5.1.3 Driver

This class is responsible for driving data to the DUT, toggling the interface signals according to the interface protocol and timing specifications.

The driver receives a transaction (sequence item) and convert it into a bit-level stream on the interface.

All protocol scenarios, including timing variations and error injection (such as parity, oversize, and illegal address), are constructed within the transaction item.

5.1.4 Monitor

The monitor samples the interface signals (in_data, in_data_vld) and reconstruct a yapp_packet transaction from the observed activity.

It verifies the correctness of the packet fields, including header, length, and parity, according to the interface specification.

The collected packet is then sent through an analysis_port to the scoreboard and coverage components for further checking and analysis.

5.1.5 Sequencer

The sequencer manages the generation and delivery of yapp_packet items from active sequences to the driver.

It supports both directed and randomized packet creation, controlling packet fields such as address, length, and parity.

Used for coordinating traffic generation to the DUT input interface.

5.1.6 Sequences

Overview

The YAPP agent sequences generate input packets to stimulate the DUT via the YAPP interface.

They include both directed and randomized scenarios, covering routing functionality, error detection, and packet integrity.

All sequences extend from yapp_base_seq, which implements UVM phase objection handling.

Defined Sequences:

- **yapp_5_packets** – Sends 5 valid packets consecutively to verify correct basic routing behavior.
- **yapp_1_seq** – Sends one packet with addr=2'b01; verifies routing to channel 1.
- **yapp_012_seq** – Sends packets with addresses 0, 1, and 2 to verify correct channel mapping.
- **yapp_111_seq** – Sends three packets with the same address to test repeated delivery to a single output channel.

- **yapp_repeat_addr_seq** – Randomly selects one address (0–2) and sends two packets to that address; validates repeated routing.
- **yapp_incr_payload_seq** – Creates a packet with incrementing payload bytes to verify data integrity and parity generation.
- **yapp_rand_seq** – Sends a random number (1–10) of randomized packets for coverage and stress verification.
- **six_yapp_seq** – A variant of the random sequence constrained to send exactly six packets.
- **yapp_exhaustive_seq** – Composite sequence executing all the above sub-sequences to achieve full directed coverage.
- **test_uvc_seq** – Iterates through all valid addresses (0–3) and lengths (1–22), randomizing parity type; verifies routing, error detection, and drop behavior.

Goal:

To verify full routing logic, parity handling, packet validation, and input interface functionality under both directed and random conditions.

5.1.7 Reference Model

Generates the expected routing behavior by decoding each input packet header and mapping it to the appropriate output channel.

Detects possible errors (illegal address, parity, oversize) and provides the expected packet classification to the scoreboard.

5.2 Channel Agent (0/1/2)

This agent is responsible for monitoring the router's outputs.

5.2.1 I/F

chan_x_if include: data_x, data_vld_x, clock, reset.

5.2.2 Config - Agent Configuration

This block defines the main configuration parameters used by the other environment components:

- Activation mode – can operate in ACTIVE or PASSIVE mode
- Channel index (0/1/2) is configurable.
- Reset behavior - defines agent behavior during and after reset.

This configuration is defined per test.

5.2.3 Driver

Applicable only when the Channel agent is configured as **ACTIVE**.

In this mode, the driver controls the suspend_x signal according to the test scenario.

When configured as **PASSIVE**, the agent does not include an active driver and only monitors the channel interface.

5.2.4 Monitor

The Channel monitor is instantiated in all configurations.

It samples the output channel signals and expects data to be transmitted according to the

output interface specification.

The received data is collected into a base transaction item, parsed into the relevant packet structure fields, and checked for protocol or communication errors.

The reconstructed packet is then published through an analysis port to upper verification layers, such as the scoreboard and coverage model, for further analysis and comparison with the expected results from the Reference Model.

5.2.5 Sequencer

The sequencer controls suspend/resume transactions for the output channels when the agent is ACTIVE.

It generates flow-control patterns to emulate FIFO full/empty behavior and test DUT backpressure handling.

Inactive in PASSIVE mode.

5.2.6 Sequences

Overview

The Channel agent sequences are used when the agent is configured in ACTIVE mode.

They control the suspend_x signal to emulate output FIFO backpressure and verify the DUT's response.

Defined Sequences:

- **channel_suspend_seq** – Actively toggles suspend_x to simulate FIFO full/empty conditions.
Verifies proper assertion of in_suspend on the input interface and recovery after release.
- **channel_idle_seq** – Keeps suspend_x deasserted to model continuous data acceptance.
Used for baseline throughput verification.
- **channel_random_suspend_seq** – Randomly toggles suspend_x at varying intervals to stress DUT flow control logic.

Goal:

To verify DUT flow control and suspend/resume mechanisms under different traffic conditions.

5.2.7 Reference Model

Predicts the expected data sequence for each output channel according to the routing decision.

Verifies that only valid packets appear on the correct channel and that dropped packets do not propagate.

5.3 HBUS Agent

This agent is responsible for the router register control interface.

5.3.1 I/F

hbus_if include: hdata[7:0], haddr[15:0], hen, hwr_rd, clock, reset.

5.3.2 Config - Agent Configuration

This block defines the main configuration parameters used by the other environment components:

- Always operate in ACTIVE mode.
- Supports read and write operations according to the specification (write = 1 clock cycle, read = 2 clock cycles).
- Reset behavior - defines agent behavior during and after reset.

This configuration is defined per test

5.3.3 Driver

This class is responsible for performing register read and write operations to the DUT, toggling the interface signals according to the register interface specifications.

The driver receives an appropriate item and translates it into the required signal sequence, ensuring that all scenarios—including configuration changes and error counter accesses—are constructed within the item and executed according to protocol timing.

5.3.4 Monitor

The monitor samples the register interface signals and expects read and write operations to be performed correctly, following the interface specifications.

The observed transactions are collected into base items, which are internally parsed to verify correct data values, response timing, and protocol compliance.

The collected items are then exported to upper verification layers for further analysis.

5.3.5 Sequencer

The sequencer issues read and write transactions to the DUT register interface.

It supports both directed and random access modes and ensures correct ordering and timing of HBUS operations.

5.3.6 Sequences

Overview

The HBUS agent sequences perform register read/write operations to configure and monitor the router DUT.

They are always run in ACTIVE mode and follow the HBUS timing protocol (write = 1 cycle, read = 2 cycles).

Defined Sequences:

- **hbus_write_seq** – Performs single and multiple register write operations to configure DUT parameters (e.g., maxpktsize, router_en).
- **hbus_read_seq** – Performs register reads to verify correct data response and protocol compliance.
- **hbus_config_seq** – Combined sequence that initializes DUT configuration registers before test start.
- **hbus_counter_check_seq** – Reads and compares internal counters (e.g., parity_err_cnt_reg, oversized_pkt_cnt_reg) to expected values after test completion.

- **hbus_illegal_access_seq** – Attempts to access invalid addresses to verify DUT ignores illegal HBUS transactions.

Goal:

To ensure HBUS register access, configuration updates, and counter operations function correctly and comply with interface timing specifications.

5.3.7 Reference Model

Models the expected register and counter behavior for HBUS read/write operations. Predicts configuration updates and internal counter values for comparison with DUT register reads.

5.4 Checkers & Scoreboards

Overview

The checkers and scoreboard provide end-to-end functional validation of the DUT by comparing actual outputs from the Channel agents with expected results generated by the Reference Model.

They also monitor internal control behavior using assertions and functional checks.

Scoreboard

- Receives expected packet information from the Reference Model and actual packets from the Channel Monitors.
- Performs packet-by-packet comparison of data, address, and parity fields.
- Tracks dropped, corrupted, or misrouted packets.
- Reports pass/fail status and summary statistics at end of simulation.
- Maintains internal counters for total packets, matched packets, and error events.

Functional Checkers

- Implement protocol-level and behavioral assertions to ensure DUT compliance.
- Verify internal control signals and error-handling responses according to the specification.

Main Checks:

1. **Routing correctness:** Packets delivered to the correct output channel based on address field.
2. **Packet length validation:** Oversized packets dropped per maxpktsize configuration.
3. **Parity check:** Detected parity errors trigger counter increment and packet drop within 1–10 cycles.
4. **Flow control:** Input in_suspend asserted when FIFO is full; data transmitted only when allowed.
5. **Data validity:** data_vld_x asserted only when valid packet data appears on the corresponding channel.

6. **Register tracking:** Internal counters (addrX_cnt_reg, oversized_pkt_cnt_reg, parity_err_cnt_reg) increment under proper conditions.
7. **Router enable behavior:** When router_en = 0, all incoming packets are dropped.
8. **HBUS protocol compliance:** Correct handshake and response timing for read/write transactions.

Assertions

Implemented at interface and system level for protocol timing and signal validity.

Each assertion triggers a UVM error message upon failure and contributes to coverage statistics.

Results Collection

The scoreboard aggregates results from all checkers, reports mismatches, and provides detailed logging for debug.

Functional coverage and error metrics are generated for regression analysis.

5.5 Error Handling

Parity Error -

- **Description:** Parity bit in the received packet does not match the calculated parity over header and payload.
- **Detection:** Monitored via parity_err_cnt_reg.
- **Behavior:** Increment parity error counter and assert error indication for 1–10 clock cycles; the faulty packet is dropped.

Oversized Packet Error -

- **Description:** Packet length field exceeds the configured maxpktsize in ctrl_reg.
- **Detection:** Monitored via oversized_pkt_cnt_reg.
- **Behavior:** Packet is dropped and the oversize counter is incremented.

Invalid Address Error -

- **Description:** Packet header address field (addr [7:6]) equals the reserved value 3.
- **Detection:** Monitored via addr3_cnt_reg (incremented upon receipt of such packet).
- **Behavior:** Packet is dropped and addr3_cnt_reg is incremented.

Router Disabled Condition -

- **Description:** Router is disabled (router_en = 0) while packets are received on the input.
- **Detection:** Verified by checking en_reg. router_en state.
- **Behavior:** All incoming packets are dropped while the router is disabled.

6 Coverage

1. Verify coverage of all valid packet lengths, including packets exceeding maxpktsize.
2. Ensure packet routing to all valid addresses and to the reserved (illegal) address.

3. Cover scenarios where packets are transmitted while the router is disabled (router_en=0).
4. Cover parity error conditions — packet with bad parity triggers error indication within 1–10 cycles.
5. Verify reset behavior during both idle and active transmission phases.
6. Cover suspend/resume events (suspend_x) for each output channel.
7. Cover enable/disable transitions of the en_reg bits controlling counter operation.
8. Cover packets with both good and bad parity values.
9. Cover suspend_x assertion while output FIFO is full.
10. Cover data_vld_x assertion during packet transmission and valid data cycles.
11. Cover read access to internal packet memory (yapp_pkt_mem) after packet reception.

7 Post Verification Notes

Several notes which raised up during work:

8 Verification issues (Opens)